Guion de trabajo tutorizado

# Hive y distribuciones Hadoop

## Plataformas Tecnológicas

*Profesor José Ignacio Merino Martín*
*Máster Big Data & Business Analytics (MBDA)*







Alumno: ÁLVARO RAMOS BERNABÉ

# ENUNCIADO DE LA ACTIVIDAD

*Guion oficial: «Trabajo Tutorizado - Hive y Distribuciones Hadoop.pdf» (Campus Virtual UPO).*

## Objetivos

• Conocer y poder trabajar con datos SQL en Databricks

## Ejercicio/Problemas

A1. Explotar datos con Hive. (3 puntos)

Descargue el fichero con los datos de las mediciones de tráfico de la ciudad de Madrid en diciembre de 2025 https://datos.madrid.es/egob/catalogo/208627-151-transporte-ptomedida-historico.zip, descomprímalo y suba el fichero a un catálogo de databricks.

Dicho fichero se encuentra en la página “Tráfico. Histórico de datos del tráfico desde 2013” del portal de datos abiertos del ayuntamiento de Madrid (https://datos.madrid.es/sites/v/index.jsp?vgnextoid=33cb30c367e78410VgnVCM1000000b205a0aRCRD&vgnextch annel=374512b9ace9f310VgnVCM100000171f5a0aRCRD)

La definición de los datos contenidos en el fichero puede encontrarla en https://datos.madrid.es/FWProjects/egob/Catalogo/Transporte/Trafico/ficheros/ Estructura_DS_Contenido_Trafico_Hi storico.pdf y el fichero de los códigos de las estaciones medidoras en https://datos.madrid.es/egob/catalogo/202468278-intensidad-trafico.csv

• Determine las fechas/horas en diciembre donde el tráfico haya alcanzado una intensidad de tráfico superior a 20.000 vehículos. ¿Cuáles son esas estaciones medidoras? (Hágalo en una misma consulta con un INNER JOIN)

• Determine la media de intensidad por mes que tiene cada una de las estaciones medidoras

• Determine el número de puntos de medición de cada una de las estaciones, ¿hay estaciones que tienen menos puntos de medición?

• Cree una tabla llamada retenciones_diciembre donde aparezca el id de la estación, el nombre de la estación, la intensidad para aquellas filas donde el valor de la ocupación de la vía sea superior al 90%

• Sobre la tabla de retenciones_diciembre haga una consulta donde aparezcan las estaciones con mayor número de instantes donde presenten una ocupación superior a ese 90%

A2. Mover datos entre datasets (3 puntos)

Descargue el fichero de Noviembre de 2019 (https://datos.madrid.es/egob/catalogo/208627-78-transporteptomedida-historico.zip), cree un volumen en el espacio workspace e importe la tabla desde ahí.

Nota: En los apuntes viene que la forma correcta sería:

CREATE TABLE traficonov (id bigint, fecha timestamp, tipo_elem string, intensidad bigint, ocupacion bigint,

carga bigint, vmed double, error string, periodo_integracion bigint ) USING CSV OPTIONS (header "true", delimiter ";") LOCATION 'dbfs:/Volumes/workspace/default/pruebavolumen/11-2019.csv';

No obstante, debido a restricciones de la cuenta free está restringido el acceso explícito a el volumen con dbfs:/ Para ello deberá de crear la tabla así

```
CREATE TABLE traficonov2019
AS SELECT * FROM read_files(
```

José Ignacio Merino Martín 1 19/1/26

'/Volumes/workspace/default/pruebavolumen/11-2019.csv', format => 'csv', sep => ';', header => true

```
);
```

Una vez creada la tabla, nos gustaría unir en otra tabla llamada “datos_dicnov” los datos de id, fecha, intensidad y ocupación de ambas tablas.

A3. Limpieza de datos en importación de un dataset (4 puntos)

Descargue y suba a un volumen el fichero https://raw.githubusercontent.com/mwaskom/seaborndata/master/taxis.csv donde se muestran un conjunto de datos de recogidas de taxis en la ciudad de Nueva York.

Se pide importar dichos datos a una tabla realizando un proceso de limpieza de los datos donde:

- Se importen datos con número de pasajeros mayor que 0
- Se importen datos con distancia mayor que 0
- Se importen datos con la cantidad total pagada mayor que 0
- Se agregue un campo adicional donde se calcule la propina (tip) como porcentaje del total de importe pagado

Una vez importado se pide:

- ¿Qué porcentaje de los viajes reciben los taxistas una propina superior a un 10%?
- ¿A qué hora del día el taxista recibe mayores propinas en valor absoluto?
- ¿A qué hora del día el taxista recibe mayores propinas en valor porcentual?
- Crea una nueva tabla donde se agregue una nueva columna e inserta en ella el valor de: o “Generoso” si la propina supera el 20%, si está entre un 1 o “Larga distancia poco Rentable” si la propia es inferior a un 5% y la distancia supera las 10 unidades o “Normal” para el resto
- Exporte dicha tabla en pandas usando df_pandas = spark.table("taxis_limpios").toPandas() y haga una representación visual sobre ella usando
- Exporta a una tabla las rutas (origen - pickup_zone y destino - dropoff_zone) donde se muestre la ruta (origen y destino), el número de viajes dado, la tarifa media y la distancia media
- Analiza dicha tabla y localiza las 5 rutas más rentables pero que tengan más de 10 viajes hechos.

## Fecha y Modo de entrega

La fecha tope para la presentación del trabajo es el 26 de Enero a las 23:59

José Ignacio Merino Martín 2 19/1/26

# RESOLUCIÓN

**Entorno de ejecución.** El guion propone Databricks Free Edition, que exige cuenta propia. Aquí las consultas se han ejecutado sobre **Apache Spark 3.5.3 con soporte Hive** (`enableHiveSupport()`), con metastore Derby y *warehouse* locales. Es el mismo motor Spark SQL que usa Databricks y el mismo dialecto HiveQL: `CREATE TABLE … STORED AS`, CTAS, `INNER JOIN`, etc. Las diferencias de sintaxis se indican donde aparecen. Por ejemplo, `read_files()` es una función exclusiva de Databricks y se sustituye por una vista temporal `USING csv` con las mismas opciones. El «volumen» `/Volumes/workspace/default/pruebavolumen` se reproduce como la carpeta `volumes/workspace/default/pruebavolumen`. Todo el SQL puede pegarse en un notebook SQL de Databricks cambiando únicamente la ruta de los ficheros.

El script `hivesql.py` ejecuta cada fichero `.sql` sentencia a sentencia y muestra el resultado y el tiempo, como la consola `spark-sql`:

```
"""Ejecuta un fichero .sql sobre Spark SQL con soporte Hive (metastore + warehouse locales)
y muestra cada sentencia y su resultado."""
import sys, time, re, os
os.environ.setdefault('JAVA_TOOL_OPTIONS','')
from pyspark.sql import SparkSession
spark = (SparkSession.builder.appName('hive-trafico').master('local[2]')
         .config('spark.sql.warehouse.dir','/home/aramber/pt09/warehouse')
         .config('spark.driver.memory','4g').config('spark.sql.shuffle.partitions','8')
         .config('spark.ui.showConsoleProgress','false')
         .config('spark.hadoop.javax.jdo.option.ConnectionURL','jdbc:derby:;databaseName=/
home/aramber/pt09/metastore_db;create=true')
         .enableHiveSupport().getOrCreate())
spark.sparkContext.setLogLevel('ERROR')
src = open(sys.argv[1], encoding='utf-8').read()
src = re.sub(r'--[^\n]*', '', src)
for st in [x.strip() for x in re.split(r';[ \t]*(?:\n|$)', src) if x.strip()]:
    print('spark-sql> ' + st.replace('\n', '\n         > '))
    t = time.time(); df = spark.sql(st)
    if df.columns and not re.match(r'(?is)^(create|insert|drop|use|set|cache|load|alter)',
st):
        n = int(os.environ.get('ROWS', '20'))
        df.show(n, truncate=False)
    else:
        df.collect()
    print(f'Time taken: {time.time()-t:.2f} seconds\n')
```

## A1. Explotar datos con Hive (tráfico de Madrid, diciembre 2025)

Se descarga `208627-151-transporte-ptomedida-historico.zip`, que contiene `12-2025.csv` (802 MB, 13,46 M de registros cada 15 minutos), y el fichero de ubicación de estaciones `202468-278-intensidad-trafico.csv`, que contiene 5 024 puntos de medida. Ambos se cargan como tablas gestionadas Parquet en la base de datos `trafico`.

```
CREATE DATABASE IF NOT EXISTS trafico;
USE trafico;
-- tabla externa sobre el CSV original (formato Hive: delimitado por ';')
CREATE TEMPORARY VIEW v_dic USING csv OPTIONS (path '/home/aramber/pt09/data/12-2025.csv',
header 'true', sep ';', inferSchema 'false');
DROP TABLE IF EXISTS trafico_dic2025;
CREATE TABLE trafico_dic2025 STORED AS PARQUET AS
SELECT CAST(id AS BIGINT) id, CAST(fecha AS TIMESTAMP) fecha, tipo_elem, CAST(intensidad AS
```

```
BIGINT) intensidad,
       CAST(ocupacion AS BIGINT) ocupacion, CAST(carga AS BIGINT) carga, CAST(vmed AS
DOUBLE) vmed, error,
       CAST(periodo_integracion AS BIGINT) periodo_integracion FROM v_dic;
CREATE TEMPORARY VIEW v_pmed USING csv OPTIONS (path
'/home/aramber/pt09/data/pmed_ubicacion.csv', header 'true', sep ';', encoding 'ISO-8859-
1');
DROP TABLE IF EXISTS estaciones;
CREATE TABLE estaciones STORED AS PARQUET AS
SELECT CAST(id AS BIGINT) id, tipo_elem, CAST(distrito AS INT) distrito, cod_cent, nombre,
CAST(latitud AS DOUBLE) latitud, CAST(longitud AS DOUBLE) longitud FROM v_pmed;
SHOW TABLES;
DESCRIBE trafico_dic2025;
SELECT COUNT(*) filas, MIN(fecha) desde, MAX(fecha) hasta, COUNT(DISTINCT id) puntos FROM
trafico_dic2025;
SELECT COUNT(*) estaciones FROM estaciones
```

```
aramber@localhost:~$ python3 hivesql.py a1_carga.sql
spark-sql> CREATE DATABASE IF NOT EXISTS trafico
Time taken: 4.13 seconds
spark-sql> USE trafico
Time taken: 0.04 seconds
spark-sql> CREATE TEMPORARY VIEW v_dic USING csv OPTIONS (path '/home/aramber/pt09/data/12-2025.csv', header 'true', sep ';', inferSchema 'false')
Time taken: 2.27 seconds
spark-sql> DROP TABLE IF EXISTS trafico_dic2025
Time taken: 0.88 seconds
spark-sql> CREATE TABLE trafico_dic2025 STORED AS PARQUET AS
         > SELECT CAST(id AS BIGINT) id, CAST(fecha AS TIMESTAMP) fecha, tipo_elem, CAST(intensidad AS BIGINT) intensidad,
         >        CAST(ocupacion AS BIGINT) ocupacion, CAST(carga AS BIGINT) carga, CAST(vmed AS DOUBLE) vmed, error,
         >        CAST(periodo_integracion AS BIGINT) periodo_integracion FROM v_dic
Time taken: 21.94 seconds
spark-sql> CREATE TEMPORARY VIEW v_pmed USING csv OPTIONS (path '/home/aramber/pt09/data/pmed_ubicacion.csv', header 'true', sep ';', encoding 'ISO-8859-1')
Time taken: 0.20 seconds
spark-sql> DROP TABLE IF EXISTS estaciones
Time taken: 0.17 seconds
spark-sql> CREATE TABLE estaciones STORED AS PARQUET AS
         > SELECT CAST(id AS BIGINT) id, tipo_elem, CAST(distrito AS INT) distrito, cod_cent, nombre, CAST(latitud AS DOUBLE) latitud, CAST(longitud AS DOUBLE)
longitud FROM v_pmed
Time taken: 0.56 seconds
spark-sql> SHOW TABLES
+---------+--------------------+-----------+
|namespace|tableName           |isTemporary|
+---------+--------------------+-----------+
|trafico  |datos_dicnov        |false      |
|trafico  |estaciones          |false      |
|trafico  |retenciones_diciembre|false      |
|trafico  |rutas               |false      |
|trafico  |taxis_categoria     |false      |
|trafico  |taxis_limpios       |false      |
|trafico  |trafico_dic2025     |false      |
|trafico  |traficonov2019      |false      |
|         |v_dic               |true       |
|         |v_pmed              |true       |
+---------+--------------------+-----------+
Time taken: 0.15 seconds
spark-sql> DESCRIBE trafico_dic2025
+------------------+---------+-------+
|col_name          |data_type|comment|
+------------------+---------+-------+
|id                |bigint   |NULL   |
|fecha             |timestamp|NULL   |
|tipo_elem         |string   |NULL   |
|intensidad        |bigint   |NULL   |
|ocupacion         |bigint   |NULL   |
|carga             |bigint   |NULL   |
|vmed              |double   |NULL   |
|error             |string   |NULL   |
|periodo_integracion|bigint   |NULL   |
+------------------+---------+-------+
Time taken: 0.07 seconds
spark-sql> SELECT COUNT(*) filas, MIN(fecha) desde, MAX(fecha) hasta, COUNT(DISTINCT id) puntos FROM trafico_dic2025
+--------+-------------------+-------------------+------+
|filas   |desde              |hasta              |puntos|
+--------+-------------------+-------------------+------+
|13459494|2025-12-01 00:00:00|2025-12-31 23:45:00|4763  |
+--------+-------------------+-------------------+------+
Time taken: 1.79 seconds
spark-sql> SELECT COUNT(*) estaciones FROM estaciones
+----------+
|estaciones|
+----------+
|5024      |
+----------+
Time taken: 0.21 seconds
```

**Figura 1.** Carga de los ficheros en el catálogo (base de datos `trafico`): 13 459 494 filas del 1 al 31/12/2025, 4 763 puntos con datos y 5 024 estaciones en el maestro.

## 1) Fechas/horas con intensidad > 20 000 vehículos y estaciones (INNER JOIN)

```
USE trafico;
SELECT t.fecha, t.id, substr(e.nombre,1,45) nombre, t.intensidad, t.error
FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id
WHERE t.intensidad > 20000
ORDER BY t.fecha;
SELECT COUNT(*) registros, COUNT(DISTINCT t.id) estaciones
FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id WHERE t.intensidad >
```

```
20000
```

```
aramber@localhost:~$ python3 hivesql.py q11.sql
spark-sql> USE trafico
Time taken: 4.03 seconds
spark-sql> SELECT t.fecha, t.id, substr(e.nombre,1,45) nombre, t.intensidad, t.error
         > FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id
         > WHERE t.intensidad > 20000
         > ORDER BY t.fecha
+-------------------+----+---------------------------------------------+----------+-----+
|fecha              |id  |nombre                                       |intensidad|error|
+-------------------+----+---------------------------------------------+----------+-----+
|2025-12-09 14:30:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|21395     |N    |
|2025-12-09 18:45:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|51355     |N    |
|2025-12-09 19:00:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|28257     |N    |
|2025-12-09 19:30:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|30882     |N    |
|2025-12-11 18:00:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|47681     |N    |
|2025-12-11 18:15:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|36103     |N    |
|2025-12-11 18:30:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|34147     |N    |
|2025-12-12 02:00:00|3751|Sor Angela de la Cruz O-E(Lateral) - Bravo Mu|46300     |N    |
|2025-12-12 02:00:00|7112|Sor Angela de la Cruz O-E(PM62083+PM62061) - |99999     |N    |
|2025-12-12 07:30:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|42876     |N    |
|2025-12-12 10:15:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|20817     |N    |
|2025-12-12 10:30:00|5432|Ginzo de Limia N-S - Fermín Caballero-Av.Ilus|34290     |N    |
+-------------------+----+---------------------------------------------+----------+-----+
only showing top 12 rows
Time taken: 4.13 seconds
spark-sql> SELECT COUNT(*) registros, COUNT(DISTINCT t.id) estaciones
         > FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id WHERE t.intensidad > 20000
+---------+----------+
|registros|estaciones|
+---------+----------+
|21       |4         |
+---------+----------+
Time taken: 1.33 seconds
```

**Figura 2.** Registros con intensidad superior a 20 000 veh/h: 21 registros en 4 estaciones.

Solo **21 instantes de 4 estaciones** superan los 20 000 veh/h. 15 corresponden al punto **5432** (Ginzo de Limia N-S; días 9, 11, 12 y 29), 4 a **6070** (Arturo Soria, máx. 23 296) y 1 a cada uno de los puntos **3751** y **7112** (Sor Ángela de la Cruz, el 12/12 a las 02:00). Son valores **anómalos**: la intensidad habitual de un punto urbano es de cientos a pocos miles de veh/h, y 99 999 es un valor centinela. Aunque el campo `error` venga como «N», deberían tratarse como lecturas erróneas del sensor en cualquier análisis posterior.

### 2) Media de intensidad por mes de cada estación

```
USE trafico;
SELECT e.id, e.nombre, date_format(t.fecha,'yyyy-MM') mes, ROUND(AVG(t.intensidad),1)
media_intensidad
FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id
WHERE t.error = 'N'
GROUP BY e.id, e.nombre, date_format(t.fecha,'yyyy-MM')
ORDER BY media_intensidad DESC
```

```
aramber@localhost:~$ python3 hivesql.py q12.sql
spark-sql> USE trafico
Time taken: 3.99 seconds
spark-sql> SELECT e.id, e.nombre, date_format(t.fecha,'yyyy-MM') mes, ROUND(AVG(t.intensidad),1) media_intensidad
         > FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id
         > WHERE t.error = 'N'
         > GROUP BY e.id, e.nombre, date_format(t.fecha,'yyyy-MM')
         > ORDER BY media_intensidad DESC
+----+----------+-------+---------------+
|id  |nombre    |mes    |media_intensidad|
+----+----------+-------+---------------+
|3819|PM10502   |2025-12|3892.8          |
|6666|PM10711   |2025-12|3640.9          |
|6846|16XC14PM01|2025-12|3468.8          |
|6679|PM11071   |2025-12|3443.3          |
|3799|PM10611   |2025-12|3412.1          |
|6733|PM20572   |2025-12|3380.0          |
|6725|PM20411   |2025-12|3363.1          |
|3820|PM10561   |2025-12|3330.4          |
|6676|PM10981   |2025-12|3324.4          |
|6842|17NC04PM01|2025-12|3301.7          |
|7122|19XC80PM01|2025-12|3295.3          |
|3490|PM10832   |2025-12|3288.1          |
+----+----------+-------+---------------+
only showing top 12 rows
Time taken: 7.81 seconds
```

**Figura 3.** Intensidad media mensual por estación (solo medidas sin error), ordenada de mayor a menor. Encabezan la lista los puntos de vías rápidas (PMxxxxx, M-30).

## 3) Número de puntos de medición de cada estación

```
USE trafico;
SELECT id, COUNT(*) puntos_medicion FROM trafico_dic2025 GROUP BY id ORDER BY
puntos_medicion ASC;
WITH c AS (SELECT id, COUNT(*) n FROM trafico_dic2025 GROUP BY id)
SELECT MAX(n) maximo, MIN(n) minimo, ROUND(AVG(n),1) media,
       SUM(CASE WHEN n < (SELECT MAX(n) FROM c) THEN 1 ELSE 0 END)
estaciones_con_menos,
       COUNT(*) total_estaciones FROM c
```

```
aramber@localhost:~$ python3 hivesql.py q13.sql
spark-sql> USE trafico
Time taken: 3.87 seconds
spark-sql> SELECT id, COUNT(*) puntos_medicion FROM trafico_dic2025 GROUP BY id ORDER BY puntos_medicion ASC
+-----+---------------+
|id   |puntos_medicion|
+-----+---------------+
|3637 |1              |
|3778 |1              |
|6505 |1              |
|6089 |1              |
|11235|1              |
|11221|1              |
|5179 |1              |
|11105|1              |
|5093 |2              |
|5068 |2              |
|6345 |2              |
|3448 |3              |
+-----+---------------+
only showing top 12 rows
Time taken: 3.92 seconds
spark-sql> WITH c AS (SELECT id, COUNT(*) n FROM trafico_dic2025 GROUP BY id)
         > SELECT MAX(n) maximo, MIN(n) minimo, ROUND(AVG(n),1) media,
         >        SUM(CASE WHEN n < (SELECT MAX(n) FROM c) THEN 1 ELSE 0 END) estaciones_con_menos,
         >        COUNT(*) total_estaciones FROM c
+------+------+------+--------------------+----------------+
|maximo|minimo|media |estaciones_con_menos|total_estaciones|
+------+------+------+--------------------+----------------+
|2966  |1     |2825.8|3563                |4763            |
+------+------+------+--------------------+----------------+
Time taken: 1.52 seconds
```

**Figura 4.** Número de registros de medición por estación y resumen estadístico.

Con registros cada 15 minutos, diciembre tendría como máximo 31 × 96 = 2 976 medidas por estación. **El máximo real es 2 966** (el dataset tiene 10 intervalos sin datos en todo el mes) y la media, 2 825,8. **Sí hay estaciones con menos puntos: 3 563 de las 4 763** están por debajo del

máximo, y algunas tienen solo 1-3 registros (ids 3637, 3778, 6505…). Son puntos dados de alta o de baja durante el mes, o con averías prolongadas del detector.

### 4) y 5) Tabla retenciones_diciembre y estaciones con más instantes de ocupación > 90 %

```
USE trafico;
DROP TABLE IF EXISTS retenciones_diciembre;
CREATE TABLE retenciones_diciembre STORED AS PARQUET AS
SELECT t.id, substr(e.nombre,1,45) nombre, t.fecha, t.intensidad, t.ocupacion
FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id
WHERE t.ocupacion > 90;
SELECT * FROM retenciones_diciembre LIMIT 5;
SELECT COUNT(*) filas, COUNT(DISTINCT id) estaciones FROM retenciones_diciembre;
SELECT id, nombre, COUNT(*) instantes_ocupacion_90 FROM retenciones_diciembre
GROUP BY id, nombre ORDER BY instantes_ocupacion_90 DESC LIMIT 10
```

```
aramber@localhost:~$ python3 hivesql.py q14.sql
spark-sql> USE trafico
Time taken: 4.09 seconds
spark-sql> DROP TABLE IF EXISTS retenciones_diciembre
Time taken: 0.88 seconds
spark-sql> CREATE TABLE retenciones_diciembre STORED AS PARQUET AS
         > SELECT t.id, substr(e.nombre,1,45) nombre, t.fecha, t.intensidad, t.ocupacion
         > FROM trafico_dic2025 t INNER JOIN estaciones e ON t.id = e.id
         > WHERE t.ocupacion > 90
Time taken: 4.05 seconds
spark-sql> SELECT * FROM retenciones_diciembre LIMIT 5
+----+---------------------------------------------+-------------------+----------+---------+
|id  |nombre                                       |fecha              |intensidad|ocupacion|
+----+---------------------------------------------+-------------------+----------+---------+
|4251|CEDACEROS S-N(CARRERA SAN JERONIMO-ALCALA)   |2025-12-17 12:45:00|105       |92       |
|4252|ALCALA E-O(CEDACEROS-PELIGROS)               |2025-12-11 12:30:00|180       |93       |
|4285|CUESTA SAN VICENTE O-E(GRAN VIA-FERRAZ)      |2025-12-11 19:00:00|156       |91       |
|4289|GRAN VIA, 59 N-S(ISABEL LA CATOLICA-SAN BERNA|2025-12-29 06:15:00|140       |97       |
|4289|GRAN VIA, 59 N-S(ISABEL LA CATOLICA-SAN BERNA|2025-12-29 07:45:00|24        |97       |
+----+---------------------------------------------+-------------------+----------+---------+
Time taken: 1.06 seconds
spark-sql> SELECT COUNT(*) filas, COUNT(DISTINCT id) estaciones FROM retenciones_diciembre
+-----+----------+
|filas|estaciones|
+-----+----------+
|14667|241       |
+-----+----------+
Time taken: 0.98 seconds
spark-sql> SELECT id, nombre, COUNT(*) instantes_ocupacion_90 FROM retenciones_diciembre
         > GROUP BY id, nombre ORDER BY instantes_ocupacion_90 DESC LIMIT 10
+----+---------------------------------------------+---------------------+
|id  |nombre                                       |instantes_ocupacion_90|
+----+---------------------------------------------+---------------------+
|6864|15RR41PM01                                   |2732                 |
|6862|15RV01PM01                                   |2732                 |
|1037|15RR22PM01                                   |2691                 |
|1036|15RT07PM01                                   |2691                 |
|4962|RAFAELA YBARRA S-N(LESACA-AV.POBLADOS)       |778                  |
|4365|QUINTANA E-O(MARTIN DE LOS HEROS-MENDIZABAL) |377                  |
|1023|14RN01PM01                                   |367                  |
|5105|OCA S-N (GRAL.RICARDOS-PL.ALMODOVAR)         |203                  |
|4629|LOPEZ DE HOYOS, 188 O-E(CLARA DEL REY-PANTOJA|176                  |
|5708|Jaén E-O - San Antonio-Bravo Murillo         |111                  |
+----+---------------------------------------------+---------------------+
Time taken: 0.53 seconds
```

**Figura 5.** Creación de `retenciones_diciembre` (id, nombre, intensidad… con ocupación > 90 %) y ranking de estaciones por número de instantes.

La tabla contiene **14 667 registros de 241 estaciones**. Las cuatro primeras (15RR41PM01, 15RV01PM01, 15RR22PM01 y 15RT07PM01, con 2 691-2 732 instantes, casi el mes entero) son puntos de la M-30 cuya ocupación permanece fija por encima del 90 %: es más probable un fallo del sensor que una retención permanente. Entre las vías urbanas destacan **Rafaela**

**Ybarra** (778 instantes, ~8 días acumulados de ocupación > 90 %), **Quintana** (377), **Oca** (203) y **López de Hoyos, 188** (176).

## A2. Mover datos entre datasets (noviembre 2019)

Se descarga `208627-78-transporte-ptomedida-historico.zip` (`11-2019.csv`, 668 MB), se copia al volumen `pruebavolumen` y se crea la tabla con CTAS. En Databricks sería `CREATE TABLE traficonov2019 AS SELECT * FROM read_files('/Volumes/workspace/default/pruebavolumen/11-2019.csv', format => 'csv', sep => ';', header => true)`. Después se unen ambos meses en `datos_dicnov` con `UNION ALL`, homogeneizando los tipos, porque en 2019 `ocupacion` se infiere como decimal.

```
USE trafico;
DROP TABLE IF EXISTS traficonov2019;
-- equivalente Spark SQL de read_files('/Volumes/workspace/default/pruebavolumen/11-
2019.csv', format=>'csv', sep=>';', header=>true)
CREATE TEMPORARY VIEW read_files_nov USING csv OPTIONS (path
'/home/aramber/pt09/volumes/workspace/default/pruebavolumen/11-2019.csv', format 'csv', sep
';', header 'true', inferSchema 'true');
CREATE TABLE traficonov2019 STORED AS PARQUET
AS SELECT * FROM read_files_nov;
SELECT * FROM traficonov2019 LIMIT 3;
SELECT COUNT(*) filas FROM traficonov2019;
DROP TABLE IF EXISTS datos_dicnov;
CREATE TABLE datos_dicnov STORED AS PARQUET AS
SELECT CAST(id AS BIGINT) id, CAST(fecha AS TIMESTAMP) fecha, CAST(intensidad AS BIGINT)
intensidad, CAST(ocupacion AS BIGINT) ocupacion FROM traficonov2019
UNION ALL
SELECT id, fecha, intensidad, ocupacion FROM trafico_dic2025;
SELECT date_format(fecha,'yyyy-MM') mes, COUNT(*) filas, COUNT(DISTINCT id) estaciones,
ROUND(AVG(intensidad),1) intensidad_media
FROM datos_dicnov GROUP BY date_format(fecha,'yyyy-MM') ORDER BY mes
```

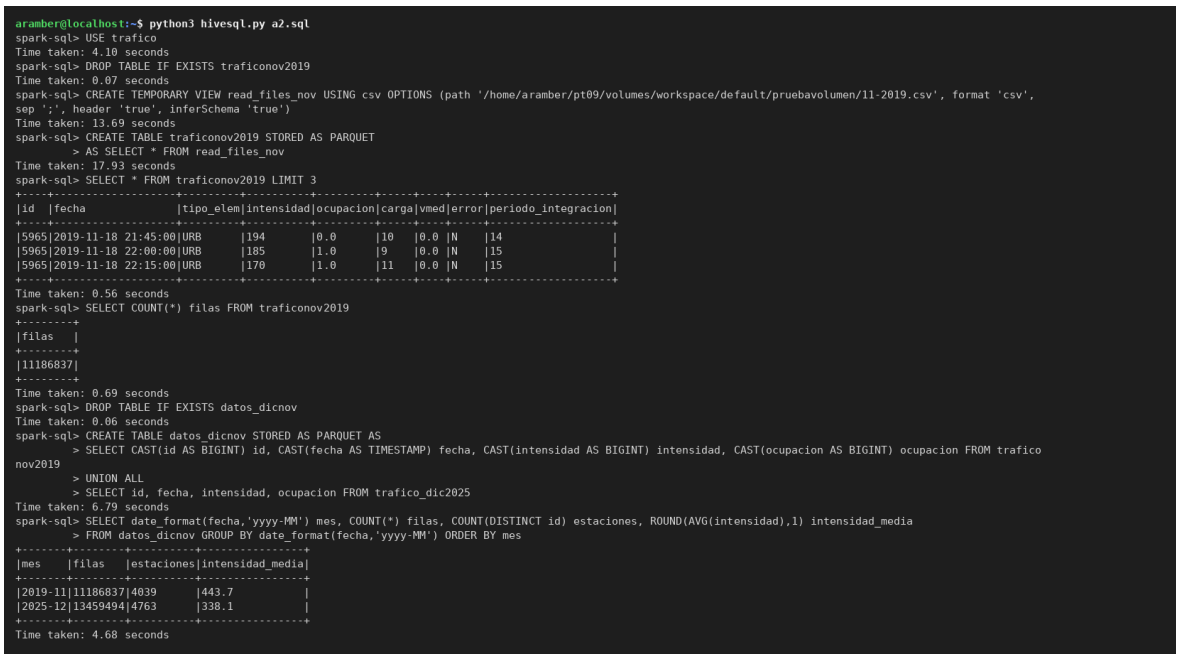


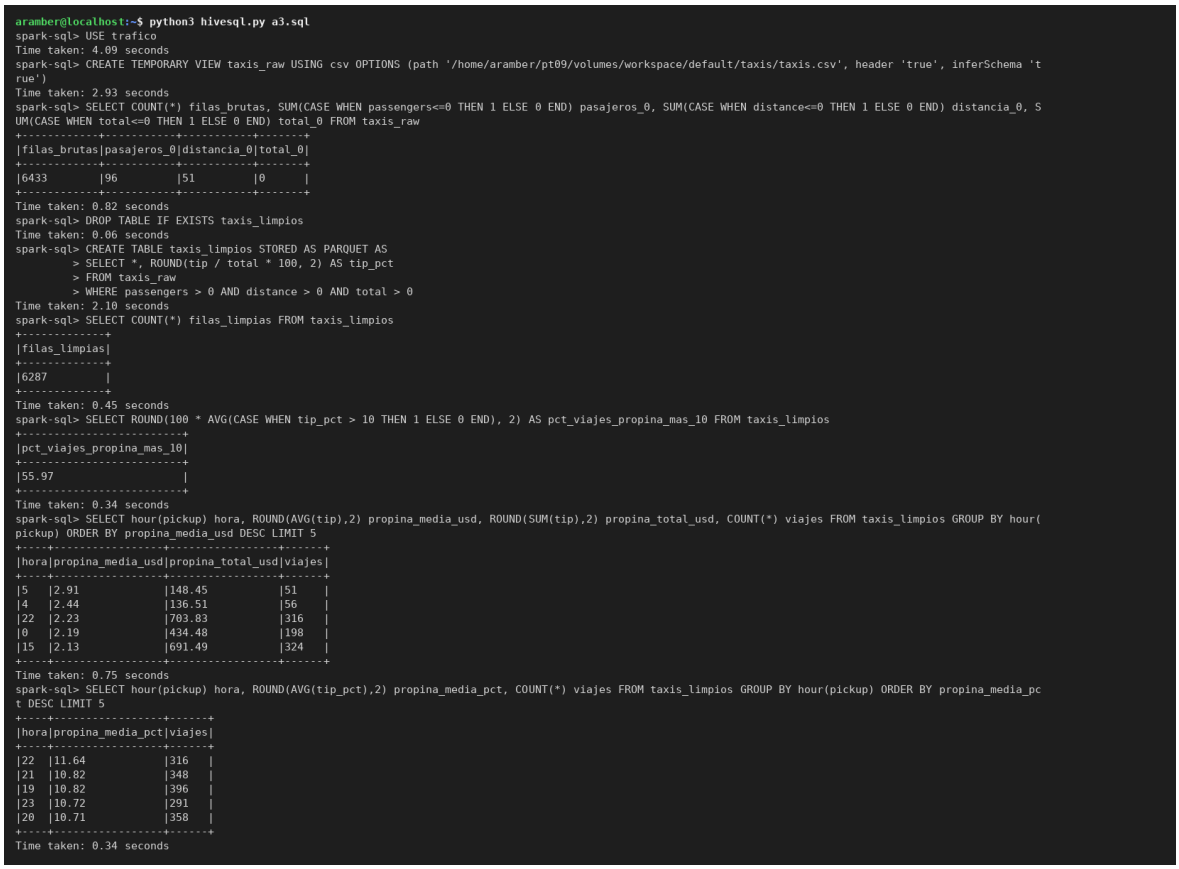

**Figura 6.** Importación de noviembre 2019 desde el volumen (11 186 837 filas) y creación de `datos_dicnov` con id, fecha, intensidad y ocupación de ambos meses.

La tabla unificada tiene 24,6 M de filas. En noviembre de 2019 había 4 039 puntos activos con una intensidad media de 443,7 veh/h. En diciembre de 2025 hay 4 763 puntos, pero la media baja a 338,1. Esa bajada se explica por la incorporación de muchos detectores urbanos de menor tráfico y por el menor volumen en periodo navideño.

## A3. Limpieza de datos en la importación (taxis de Nueva York)

`taxis.csv` (seaborn-data) se descarga al volumen `taxis`. La limpieza se hace en la propia sentencia de importación: filtros `passengers > 0`, `distance > 0` y `total > 0`, y cálculo del campo **`tip_pct` = tip / total × 100**.

```
USE trafico;
CREATE TEMPORARY VIEW taxis_raw USING csv OPTIONS (path
'/home/aramber/pt09/volumes/workspace/default/taxis/taxis.csv', header 'true', inferSchema
'true');
SELECT COUNT(*) filas_brutas, SUM(CASE WHEN passengers<=0 THEN 1 ELSE 0 END) pasajeros_0,
SUM(CASE WHEN distance<=0 THEN 1 ELSE 0 END) distancia_0, SUM(CASE WHEN total<=0 THEN 1
ELSE 0 END) total_0 FROM taxis_raw;
DROP TABLE IF EXISTS taxis_limpios;
CREATE TABLE taxis_limpios STORED AS PARQUET AS
SELECT *, ROUND(tip / total * 100, 2) AS tip_pct
FROM taxis_raw
WHERE passengers > 0 AND distance > 0 AND total > 0;
SELECT COUNT(*) filas_limpias FROM taxis_limpios;
SELECT ROUND(100 * AVG(CASE WHEN tip_pct > 10 THEN 1 ELSE 0 END), 2) AS
pct_viajes_propina_mas_10 FROM taxis_limpios;
SELECT hour(pickup) hora, ROUND(AVG(tip),2) propina_media_usd, ROUND(SUM(tip),2)
propina_total_usd, COUNT(*) viajes FROM taxis_limpios GROUP BY hour(pickup) ORDER BY
propina_media_usd DESC LIMIT 5;
SELECT hour(pickup) hora, ROUND(AVG(tip_pct),2) propina_media_pct, COUNT(*) viajes FROM
taxis_limpios GROUP BY hour(pickup) ORDER BY propina_media_pct DESC LIMIT 5;
DROP TABLE IF EXISTS taxis_categoria;
CREATE TABLE taxis_categoria STORED AS PARQUET AS
SELECT *, CASE WHEN tip_pct > 20 THEN 'Generoso'
               WHEN tip_pct < 5 AND distance > 10 THEN 'Larga distancia poco Rentable'
               ELSE 'Normal' END AS categoria
FROM taxis_limpios;
SELECT categoria, COUNT(*) viajes FROM taxis_categoria GROUP BY categoria ORDER BY viajes
DESC;
DROP TABLE IF EXISTS rutas;
CREATE TABLE rutas STORED AS PARQUET AS
SELECT pickup_zone AS origen, dropoff_zone AS destino, COUNT(*) AS num_viajes,
       ROUND(AVG(fare),2) AS tarifa_media, ROUND(AVG(distance),2) AS distancia_media,
       ROUND(AVG(fare)/AVG(distance),2) AS tarifa_por_milla
FROM taxis_limpios WHERE pickup_zone IS NOT NULL AND dropoff_zone IS NOT NULL
GROUP BY pickup_zone, dropoff_zone;
SELECT COUNT(*) rutas FROM rutas;
SELECT * FROM rutas WHERE num_viajes > 10 ORDER BY tarifa_por_milla DESC LIMIT 5;
SELECT * FROM rutas WHERE num_viajes > 10 ORDER BY tarifa_media DESC LIMIT 5
```



```
aramber@localhost:~$ python3 hivesql.py a3.sql
spark-sql> USE trafico
Time taken: 4.09 seconds
spark-sql> CREATE TEMPORARY VIEW taxis_raw USING csv OPTIONS (path '/home/aramber/pt09/volumes/workspace/default/taxis/taxis.csv', header 'true', inferSchema 't
rue')
Time taken: 2.93 seconds
spark-sql> SELECT COUNT(*) filas_brutas, SUM(CASE WHEN passengers<=0 THEN 1 ELSE 0 END) pasajeros_0, SUM(CASE WHEN distance<=0 THEN 1 ELSE 0 END) distancia_0, S
UM(CASE WHEN total<=0 THEN 1 ELSE 0 END) total_0 FROM taxis_raw
+------------+-----------+-----------+-------+
|filas_brutas|pasajeros_0|distancia_0|total_0|
+------------+-----------+-----------+-------+
|6433        |96         |51         |0      |
+------------+-----------+-----------+-------+
Time taken: 0.82 seconds
spark-sql> DROP TABLE IF EXISTS taxis_limpios
Time taken: 0.06 seconds
spark-sql> CREATE TABLE taxis_limpios STORED AS PARQUET AS
         > SELECT *, ROUND(tip / total * 100, 2) AS tip_pct
         > FROM taxis_raw
         > WHERE passengers > 0 AND distance > 0 AND total > 0
Time taken: 2.10 seconds
spark-sql> SELECT COUNT(*) filas_limpias FROM taxis_limpios
+-------------+
|filas_limpias|
+-------------+
|6287         |
+-------------+
Time taken: 0.45 seconds
spark-sql> SELECT ROUND(100 * AVG(CASE WHEN tip_pct > 10 THEN 1 ELSE 0 END), 2) AS pct_viajes_propina_mas_10 FROM taxis_limpios
+------------------------+
|pct_viajes_propina_mas_10|
+------------------------+
|55.97                   |
+------------------------+
Time taken: 0.34 seconds
spark-sql> SELECT hour(pickup) hora, ROUND(AVG(tip),2) propina_media_usd, ROUND(SUM(tip),2) propina_total_usd, COUNT(*) viajes FROM taxis_limpios GROUP BY hour(
pickup) ORDER BY propina_media_usd DESC LIMIT 5
+----+-----------------+-----------------+------+
|hora|propina_media_usd|propina_total_usd|viajes|
+----+-----------------+-----------------+------+
|5   |2.91             |148.45           |51    |
|4   |2.44             |136.51           |56    |
|22  |2.23             |703.83           |316   |
|0   |2.19             |434.48           |198   |
|15  |2.13             |691.49           |324   |
+----+-----------------+-----------------+------+
Time taken: 0.75 seconds
spark-sql> SELECT hour(pickup) hora, ROUND(AVG(tip_pct),2) propina_media_pct, COUNT(*) viajes FROM taxis_limpios GROUP BY hour(pickup) ORDER BY propina_media_pc
t DESC LIMIT 5
+----+-----------------+------+
|hora|propina_media_pct|viajes|
+----+-----------------+------+
|22  |11.64            |316   |
|21  |10.82            |348   |
|19  |10.82            |396   |
|23  |10.72            |291   |
|20  |10.71            |358   |
+----+-----------------+------+
Time taken: 0.34 seconds
```

**Figura 7.** Filas brutas y descartadas, tabla `taxis_limpios`, porcentaje de viajes con propina > 10 % y horas con mayor propina absoluta y porcentual.

- **Limpieza:** de 6 433 viajes se descartan 96 sin pasajeros y 51 sin distancia (ninguno con total ≤ 0). Quedan **6 287 viajes limpios**.
- **Viajes con propina superior al 10 %: 55,97 %.**
- **Hora con mayores propinas en valor absoluto:** las **5:00** (2,91 USD de media), seguida de las 4:00. Son trayectos nocturnos largos, a menudo aeropuertos, con pocos viajes. En volumen total, la hora que más dinero deja en propinas es las 22:00 (703,83 USD).
- **Hora con mayores propinas en porcentaje:** las **22:00** (11,64 % de media), seguida de las 19:00-21:00 y las 23:00: la franja de ocio nocturno.

```
spark-sql> DROP TABLE IF EXISTS taxis_categoria
Time taken: 0.05 seconds
spark-sql> CREATE TABLE taxis_categoria STORED AS PARQUET AS
         > SELECT *, CASE WHEN tip_pct > 20 THEN 'Generoso'
         >                WHEN tip_pct < 5 AND distance > 10 THEN 'Larga distancia poco Rentable'
         >                ELSE 'Normal' END AS categoria
         > FROM taxis_limpios
Time taken: 0.59 seconds
spark-sql> SELECT categoria, COUNT(*) viajes FROM taxis_categoria GROUP BY categoria ORDER BY viajes DESC
+---------------------------+------+
|categoria                  |viajes|
+---------------------------+------+
|Normal                     |5819  |
|Generoso                   |297   |
|Larga distancia poco Rentable|171   |
+---------------------------+------+
Time taken: 0.41 seconds
spark-sql> DROP TABLE IF EXISTS rutas
Time taken: 0.07 seconds
spark-sql> CREATE TABLE rutas STORED AS PARQUET AS
         > SELECT pickup_zone AS origen, dropoff_zone AS destino, COUNT(*) AS num_viajes,
         >        ROUND(AVG(fare),2) AS tarifa_media, ROUND(AVG(distance),2) AS distancia_media,
         >        ROUND(AVG(fare)/AVG(distance),2) AS tarifa_por_milla
         > FROM taxis_limpios WHERE pickup_zone IS NOT NULL AND dropoff_zone IS NOT NULL
         > GROUP BY pickup_zone, dropoff_zone
Time taken: 0.74 seconds
spark-sql> SELECT COUNT(*) rutas FROM rutas
+-----+
|rutas|
+-----+
|2712 |
+-----+
Time taken: 0.22 seconds
spark-sql> SELECT * FROM rutas WHERE num_viajes > 10 ORDER BY tarifa_por_milla DESC LIMIT 5
+--------------------+------------------------+----------+------------+--------------+----------------+
|origen              |destino                 |num_viajes|tarifa_media|distancia_media|tarifa_por_milla|
+--------------------+------------------------+----------+------------+--------------+----------------+
|East Village        |East Village            |13        |4.38        |0.45           |9.83            |
|Clinton East        |Times Sq/Theatre District|12        |5.29        |0.56           |9.42            |
|Midtown East        |Midtown Center          |11        |7.09        |0.78           |9.08            |
|Upper East Side South|Upper East Side South   |14        |5.43        |0.62           |8.83            |
|Midtown South       |Murray Hill             |11        |5.36        |0.64           |8.4             |
+--------------------+------------------------+----------+------------+--------------+----------------+
Time taken: 0.22 seconds
spark-sql> SELECT * FROM rutas WHERE num_viajes > 10 ORDER BY tarifa_media DESC LIMIT 5
+--------------------+---------------------------+----------+------------+--------------+----------------+
|origen              |destino                    |num_viajes|tarifa_media|distancia_media|tarifa_por_milla|
+--------------------+---------------------------+----------+------------+--------------+----------------+
|Upper East Side North|Midtown East               |17        |10.56       |1.76           |5.99            |
|Gramercy            |Midtown Center             |11        |9.91        |1.74           |5.71            |
|Midtown Center      |Upper East Side North      |14        |9.86        |1.92           |5.14            |
|Midtown Center      |Penn Station/Madison Sq West|12        |9.08        |1.2            |7.58            |
|Midtown Center      |Union Sq                   |11        |9.05        |1.5            |6.03            |
+--------------------+---------------------------+----------+------------+--------------+----------------+
Time taken: 0.12 seconds
```

**Figura 8.** Tabla `taxis_categoria` con la nueva columna, tabla `rutas` (origen, destino, nº de viajes, tarifa y distancia medias) y top-5 de rutas rentables con más de 10 viajes.

**Nueva columna `categoria`:** «Generoso» si la propina supera el 20 %, «Larga distancia poco Rentable» si la propina es inferior al 5 % y la distancia supera 10 millas, y «Normal» en el resto. Resultado: 5 819 Normal, 297 Generoso y 171 Larga distancia poco Rentable.

### Exportación a pandas y representación visual:

```
import os; os.environ['JAVA_TOOL_OPTIONS']=''
from pyspark.sql import SparkSession
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
spark = (SparkSession.builder.master('local[2]').config('spark.sql.warehouse.dir','/home/
aramber/pt09/warehouse')
         .config('spark.hadoop.javax.jdo.option.ConnectionURL','jdbc:derby:;databaseName=/
home/aramber/pt09/metastore_db;create=true')
         .enableHiveSupport().getOrCreate()); spark.sparkContext.setLogLevel('ERROR')
spark.sql('USE trafico')
df_pandas = spark.table("taxis_limpios").toPandas()
df_cat = spark.table("taxis_categoria").toPandas()
print(type(df_pandas), df_pandas.shape)
print(df_pandas[['pickup','distance','fare','tip','total','tip_pct','pickup_borough']].head
())
df_pandas['hora'] = df_pandas.pickup.dt.hour
```

```
fig, ax = plt.subplots(1, 3, figsize=(16, 4.5))
df_pandas.groupby('hora').tip_pct.mean().plot.bar(ax=ax[0], color='#4A7EBB');
ax[0].set_title('Propina media (% del total) por hora'); ax[0].set_ylabel('%')
df_cat.categoria.value_counts().plot.pie(ax=ax[1], autopct='%1.1f%%', startangle=90);
ax[1].set_ylabel(''); ax[1].set_title('Viajes por categoría')
d = df_pandas[df_pandas.distance < 25]
ax[2].scatter(d.distance, d.tip, s=6, alpha=.35, c=(d.payment=='credit
card').map({True:'#2E8B57', False:'#D2691E'}))
ax[2].set_xlabel('distancia (millas)'); ax[2].set_ylabel('propina (USD)');
ax[2].set_title('Propina vs distancia (verde: tarjeta, naranja: efectivo)')
plt.tight_layout(); plt.savefig('/opt/w2/shots/pt09_a3_plot.png', dpi=110); print('gráfico
guardado: pt09_a3_plot.png')
print('Propina media por forma de pago:');
print(df_pandas.groupby('payment').tip_pct.mean().round(2))
```

```
aramber@localhost:~$ python3 a3_pandas.py

<class 'pandas.DataFrame'> (6287, 15)
              pickup  distance  fare   tip  total  tip_pct pickup_borough
0 2019-03-23 20:21:09      1.60   7.0  2.15  12.95    16.60      Manhattan
1 2019-03-04 16:11:55      0.79   5.0  0.00   9.30     0.00      Manhattan
2 2019-03-27 17:53:01      1.37   7.5  2.36  14.16    16.67      Manhattan
3 2019-03-10 01:23:59      7.70  27.0  6.15  36.95    16.64      Manhattan
4 2019-03-30 13:27:42      2.16   9.0  1.10  13.40     8.21      Manhattan
gráfico guardado: pt09_a3_plot.png
Propina media por forma de pago:
payment
cash           0.00
credit card    14.11
Name: tip_pct, dtype: float64
```

**Figura 9.** Ejecución: `spark.table("taxis_limpios").toPandas()` devuelve un DataFrame de pandas de 6 287 × 15.




**Figura 10.** Representación visual con pandas y matplotlib: propina media por hora, reparto por categoría y propina frente a distancia.

El gráfico de dispersión revela un sesgo importante del dataset: **los pagos en efectivo registran siempre propina 0** (tip_pct medio de 0 % frente a 14,1 % con tarjeta), porque la propina en efectivo no queda registrada en el taxímetro. Parte de la categoría «Larga distancia poco Rentable» procede de este efecto y no de clientes poco generosos.

**Tabla de rutas y las 5 rutas más rentables con más de 10 viajes.** La tabla `rutas` agrupa 2 712 pares origen-destino. Como «rentabilidad» se usa la **tarifa por milla** (tarifa media / distancia media), que mide cuánto ingresa el taxista por distancia recorrida:

| Origen | Destino | Viajes | Tarifa media | Distancia media | USD/milla |
|---|---|---|---|---|---|
| East Village | East Village | 13 | 4,38 | 0,45 | 9,83 |
| Clinton East | Times Sq/Theatre District | 12 | 5,29 | 0,56 | 9,42 |
| Midtown East | Midtown Center | 11 | 7,09 | 0,78 | 9,08 |

| | | | | | |
|---|---|---|---|---|---|
| Upper East Side South | Upper East Side South | 14 | 5,43 | 0,62 | 8,83 |
| Midtown South | Murray Hill | 11 | 5,36 | 0,64 | 8,40 |

Las rutas más rentables por milla son **trayectos cortos dentro de Manhattan**: la bajada de bandera pesa mucho sobre una distancia pequeña y, además, permiten encadenar servicios. Si se considera la rentabilidad por viaje (tarifa media), destacan Upper East Side North → Midtown East (10,56 USD) y Gramercy → Midtown Center (9,91 USD).

## Referencias

Portal de Datos Abiertos del Ayuntamiento de Madrid – Tráfico. Histórico de datos del tráfico desde 2013: https://datos.madrid.es

Apache Spark SQL – Hive Tables: https://spark.apache.org/docs/3.5.3/sql-data-sources-hive-tables.html

Databricks – read_files table-valued function: https://docs.databricks.com/en/sql/language-manual/functions/read_files.html

Waskom, M. seaborn-data: taxis.csv – https://github.com/mwaskom/seaborn-data